SIMATS ENGINEERING

CO2_AT2_ REVERSE ENGINEERING TASK – ANSWERS

Student Name: PRADEEP.S

Registration Number: 192521024

Course Code: CSA3711

Course name: SOFTWARE TESTING

1. ATM Withdrawal Module – Reverse Engineering Black-Box Tests

System decomposition:

1. Input interface: accepts account details and withdrawal amount.
2. Validation component: checks format, positive amount, and permitted withdrawal limits.
3. Account component: checks account status and available balance.
4. Transaction component: debits the account and records the transaction.
5. Response component: displays success or an appropriate error.

Likely equivalence classes: (a) valid positive amount within the allowed limit and available balance; (b) amount below the minimum; (c) amount above the maximum; (d) zero or negative amount; (e) amount greater than available balance; (f) invalid, blank, or non-numeric input; and (g) invalid/blocked account. Exact limits must be obtained from requirements or observed behavior.

Boundary tests should use minimum−1, minimum, minimum+1, maximum−1, maximum, and maximum+1 in the smallest supported currency unit. Also test the exact available balance, balance+1, a cash amount not supported by the ATM denomination, and repeated withdrawal requests.

The partial tests likely used basic positive and negative examples, perhaps ECP, but boundary failures suggest BVA was incomplete. Reconstructed tests should check output message, balance before and after, transaction history, cash-dispense result, and rollback on a failure. Include network interruption and duplicate-request tests to ensure an unsuccessful or repeated request does not debit the account incorrectly. This design is stronger because it covers valid classes, invalid classes, boundaries, and transaction integrity.

2. Reconstructing White-Box Testing from Execution Traces

The question refers to code snippets and execution traces, but no actual snippets or traces are included in the supplied paper. Therefore, an exact CFG and numeric coverage calculation cannot be reconstructed from the source document alone. The following is a suitable conceptual method.

Construct the CFG by representing each sequential statement as a node, each if/else or switch as a decision with outgoing edges, each loop as an entry/body/back edge/exit, and each return or exception as an exit edge. Label every decision and loop so the trace data can be mapped to the graph.

For a simple example: Entry → Validate input → [Valid?]. False → Error → Exit. True → [Loop condition?]. True → Process item → Update counter → return to loop condition. False → Finalize → Exit. Independent paths include invalid input; valid input with zero loop iterations if allowed; valid input with one iteration; and valid input with multiple iterations.

Evaluate statement coverage, branch coverage, condition coverage, and path coverage using the actual trace evidence. A trace that executes each statement may still omit one decision outcome. Calculate cyclomatic complexity as $V(G) = E - N + 2P$, where E is edges, N is nodes, and P is connected components; for one connected graph, $V(G) = E - N + 2$. Design a basis set of independent paths, test each branch outcome, and test loop boundaries (zero, one, two, and maximum supported iterations). Full path coverage may be infeasible when loops produce unbounded paths, so state the practical coverage criterion used and document any exclusions.

3. Login and Transaction System – Hybrid Reverse-Engineered Strategy

Decompose the system into login input validation, credential verification, session creation, authorization, transaction validation, balance/limit checks, transaction execution, audit logging, and error handling.

Defect patterns can suggest likely techniques, but cannot prove which techniques were originally used. Failures at exact limits suggest inadequate BVA; failures for invalid input types suggest weak ECP; defects triggered by a particular decision sequence suggest missing path or state tests; and failures after timeout/retry suggest insufficient integration and recovery tests.

Black-box ECP should include valid credentials, wrong password, unknown account, blank fields, malformed input, valid/invalid OTP if applicable, authorized/unauthorized transaction, and sufficient/insufficient balance. BVA should test transaction minimum and maximum, available balance and balance+1, OTP expiry boundaries, and attempt-count limits. Decision tables should combine account status, credentials, authorization, balance, and transaction type.

White-box testing should build CFGs for authentication and transaction decision logic. Test both outcomes of each decision and independent paths for successful login, invalid credentials, locked account, unauthorized operation, insufficient balance, failed transaction, and recovery/rollback. Add security cases for rate limiting, session expiry, replay/duplicate submissions, and safe error handling.

Use traceability to map each requirement and high-risk code path to tests. Measure branch/path coverage where feasible, run integration and regression tests, and compare logs with expected state changes. The integrated strategy detects input, logic, and interaction defects more effectively than relying on one unknown testing technique.

4. Inventory Management – Inferring Boundaries from Logs

Break the system into stock input, stock validation, inventory update, low-stock threshold check, alert generation, and transaction logging. The supplied assessment mentions successful and failed logs but does not include the actual log entries or threshold values, so the precise hidden rule cannot be inferred. The following tests are appropriate for reconstructing it.

Define equivalence classes for stock quantity: valid positive quantity; zero quantity; negative quantity; non-integer quantity if only whole units are permitted; and malformed or missing quantity. For an update operation, distinguish stock addition from stock removal and confirm whether removal is rejected when it would make stock negative.

If the low-stock threshold is T, test stock levels $T-1$, T, and $T+1$. This determines whether the alert rule is stock $< T$ or stock $\leq T$. Also test zero stock, restocking from below T to above T, and multiple updates that cross the threshold in both directions.

Check that a failed update leaves stock unchanged, successful updates are atomic, and alerts are neither missed nor duplicated. Include concurrent updates, duplicate requests, and database errors. Reconstruct the rule by comparing input, prior stock, requested change, resulting stock, and alert outcome in the logs. The revised black-box design uses ECP, BVA, and state/decision combinations to target the likely gaps while making assumptions explicit.

5. Library Management EXE – Resource Profile and Path Testing

A compiled executable and CPU/memory profile can reveal performance symptoms, but they cannot uniquely identify source-code branches or prove which loop caused a spike. The actual profile and executable behavior are not included in the assessment PDF, so the CFG below is conceptual.

Decompose the module into request validation, book/member lookup, availability checking, borrowing/return processing, database/file access, and report or response generation. A conceptual CFG is: Entry → Validate request → [Valid?]; invalid → error/exit; valid → search for book → [Book found?]; no → not-found/exit; yes → check availability → [Available?]; no → unavailable/exit; yes → update loan record → update inventory → confirm/exit. A loop may exist around search, record scanning, or report generation.

Map CPU spikes to repeated processing and memory spikes to large collections, repeated allocations, or retained objects as hypotheses to investigate. Instrument the program, collect call stacks/profiling data if available, and confirm hotspots before assigning them to CFG nodes. Build path tests for each decision outcome, empty and large datasets, missing records, repeated searches, and error/rollback paths. Exercise loop counts of zero, one, two, typical, and maximum supported sizes.

Measure execution time, peak memory, leaks, and response correctness. Add stress and endurance tests and verify resource use returns to an acceptable level after operations. This strategy targets expensive paths while avoiding unsupported claims about unseen source code.

6. Mobile Payment Gateway – State Transition Testing

Model the transaction states as Pending, Success, Reversal, and Timeout. The exact JSON examples are not included in the assessment paper, so transition triggers and field constraints must be confirmed against the API specification.

A conceptual state diagram is:
Pending → Success when payment confirmation is received.
Pending → Timeout when the allowed response period expires.
Pending → Reversal when the provider or bank reverses the transaction according to policy.
Timeout → Success only if late confirmation is explicitly supported and safely reconciled; otherwise it should be rejected or handled through reconciliation.
Success → Reversal if a valid reversal is permitted.
Success should not be changed back to Pending by a duplicate or delayed request.
Reversal and Timeout handling should be idempotent and must not cause multiple refunds or debits.

Test every legal transition, every forbidden transition, duplicate callbacks, delayed/out-of-order responses, malformed JSON, missing transaction ID, invalid amount/currency, and repeated requests. Verify status, ledger entries, timestamps, HTTP response, and idempotency key behavior. State-

transition coverage tests each state and transition; transition-pair coverage checks sequences of two transitions.

Check invariants: a transaction must not be charged twice, a reversal must be traceable to the original transaction, and the final ledger state must match the reported status. Include timeout recovery and reconciliation tests. This state-based suite is more suitable than testing each API response in isolation because defects often occur in event sequences.

7. Encrypted Communication Module – Fuzzing and Robust BVA

Decompose the module into input parsing/validation, length and format checking, key/nonce handling, encryption, output encoding, and error handling. Since the actual interface and implementation are not provided, exact field sizes and valid formats must be taken from the API specification.

For each field, define minimum and maximum permitted lengths and test minimum−1, minimum, minimum+1, maximum−1, maximum, and maximum+1. Include empty input, oversized input, malformed encoding, unexpected Unicode, embedded nulls, invalid key/nonce lengths, missing fields, extra fields, and wrong data types. Fuzzing can generate malformed and unexpected combinations, while BVA systematically checks known limits.

Use safe, authorized testing in an isolated environment. Verify that oversized or malformed data is rejected safely, the process does not crash, memory remains bounded, and errors do not disclose sensitive information. Confirm that input length is validated before copying or processing data, and that cryptographic parameters are validated according to the chosen algorithm.

BVA targets known length boundaries; fuzzing explores unexpected input structures. Together they can reveal parsing errors, unsafe memory handling, and injection risks. However, testing alone cannot guarantee prevention of buffer overflows or injection. Secure coding, memory-safe APIs, code review, static analysis, and dependency checks are also required.

8. Insurance Premium Calculator – Branch and Path Testing

100% statement coverage means every executable statement ran, but it does not prove that every outcome of a multi-condition decision was tested. A test may execute the calculation statement while never exercising the false branch or important combinations of conditions.

The actual source code is not supplied, so the precise CFG and independent paths cannot be listed. A conceptual example is: Entry → Validate age and policy type → [Age eligible?] → [Policy active?] →

[Risk category?] → Calculate premium or reject request → Exit. Each decision must have true and false outgoing edges, and the graph should match the real source.

Reconstruct tests from the conditions. If premium eligibility depends on AgeEligible AND PolicyActive AND RiskAcceptable, test a successful case, then cases where each condition independently becomes false while the others remain true. Also test every branch outcome, boundary ages, policy start/expiry dates, risk-category boundaries, invalid values, and calculation rounding.

Calculate cyclomatic complexity $V(G) = E - N + 2P$ to estimate the number of linearly independent paths. Achieve 100% branch coverage and basis-path coverage where feasible. Be careful: 100% unrestricted path coverage is generally impossible when loops or many conditions create numerous paths, and branch coverage does not guarantee all condition combinations. For high-risk decisions, use condition coverage or MC/DC as required.

Compare calculated premiums against approved examples and an independent reference calculation. The revised suite reduces risk by testing decision outcomes and condition interactions rather than relying on statement coverage alone.

9. Cloud File Synchronization – ECP, BVA and Stress Testing

Decompose the synchronization process into file discovery, metadata comparison, upload/download, conflict resolution, network retry, temporary storage, and final commit. The defect reports mentioned in the question are not included in the supplied paper, so the likely original partitions can only be hypothesized.

ECP for file size should include empty files, normal files, maximum supported files, and oversized files. File-type classes should include supported formats, unsupported formats, files with unusual names/extensions, and files with special characters if permitted. Network classes should include stable fast connection, slow connection, intermittent connection, disconnection, and reconnection. Include read-only files, duplicate names, and concurrent edits where relevant.

BVA should test zero bytes, one byte, the maximum allowed size, and just above the maximum. For chunked transfer, test chunk-size−1, exact chunk size, chunk-size+1, and files with a final partial chunk. Stress testing should cover many files, large aggregate data, repeated synchronization, and network interruptions during transfer.

Verify integrity with checksums or hashes, compare source and destination file counts and content, and confirm that partial transfers are not reported as complete. Test retry and resume behavior, conflict resolution, and duplicate-event handling. ECP provides representative categories, BVA targets size

limits, and stress testing exposes resource and concurrency issues. Together they reduce data-loss risk when paired with atomic writes, durable checkpoints, and recovery mechanisms.

10. E-Commerce Search and Filter – Decision Table Testing

The SQL query and resulting dataset can be used to infer how search filters are combined, but the actual queries and outputs are not included in the assessment paper. Therefore, the table below is a template; substitute the real fields and observed results when available.

Decompose the module into parameter validation, search-term matching, category filtering, price filtering, availability filtering, sorting, pagination, and query construction. Determine whether filters are joined with AND (all selected conditions must match), OR (any selected condition may match), and NOT (exclude matching records). Also inspect behavior for empty parameters and default values.

A sample decision table can use conditions: search term matches (Y/N), category matches (Y/N), price in range (Y/N), and item available (Y/N). Rule A: all required conditions are true → include item. Rule B: any condition connected by AND is false → exclude item. Rule C: if a category group is specified as OR, include an item matching either selected category. Rule D: if an exclusion filter matches, exclude the item. Rule E: no filters supplied → return the default dataset or the specified validation message.

Test one filter at a time, all filters together, AND/OR combinations, exclusion filters, contradictory filters, empty result sets, missing values, special characters, invalid ranges, sorting, and pagination. Use representative combinations and pairwise testing to control test count, while explicitly testing high-risk logical combinations. Verify results against a manually calculated expected dataset and confirm that parameterized queries are used. This provides a structured check of the inferred decision logic and query completeness.

Key Terms for Revision

- **Reverse engineering:** Inferring a system's structure or rules from observed behavior, logs, traces, outputs, or executable behavior.
- **ECP:** Groups inputs into classes expected to behave similarly and selects representative values.
- **BVA:** Tests values at and immediately around input limits.
- **CFG:** A graph of program statements, decisions, loops, and control transfers.
- **Cyclomatic complexity:** $V(G) = E - N + 2P$; estimates the number of linearly independent paths.
- **Branch coverage:** Exercises every decision outcome at least once.
- **Basis-path testing:** Tests a set of linearly independent execution paths.

- **State-transition testing:** Tests states, events, legal transitions, and forbidden transitions.
- **Decision table testing:** Maps combinations of conditions to expected actions or outputs.
- **Fuzzing:** Supplies many unexpected or malformed inputs to expose robustness and validation defects.

Before Submission

- Fill in your name and registration number.
- Where your instructor provides actual logs, code snippets, JSON responses, SQL queries, or profiles, update the conceptual examples to match that evidence.
- Check exact limits, state rules, and expected outputs against the given system requirements.
- Read and understand the answers, then complete the original-work declaration and signature as required.